



Đặt phòng Vũng Tàu Stay

Thiết kế & Triển khai CSDL - Kiến trúc Thick-Database (MySQL InnoDB & Python Flask)

Bối cảnh & Lý do

- ✓ **Thực trạng:** Lưu lượng khách tăng đột biến dịp lễ đòi hỏi hệ thống xử lý hàng nghìn giao dịch đồng thời.
- ✓ **Vấn đề cốt lõi:** Tránh tình trạng *Overbooking* (đặt trùng phòng) gây thiệt hại nghiêm trọng về uy tín và tài chính.
- ✓ **Giải pháp kiến trúc:** Áp dụng hướng đi **Thick-Database**, đẩy toàn bộ logic nghiệp vụ, giao tác và đồng bộ hóa xuống thẳng CSDL để giải quyết triệt để rủi ro.



Tổng quan Hệ thống



Người dùng

Khách hàng sử dụng giao diện đặt phòng trực tuyến. Lễ tân/Admin sử dụng giao diện quản trị để theo dõi, xử lý đơn và vận hành.



Quy trình lõi

Tìm và Lọc phòng → Đặt phòng an toàn → Quản lý Check-in → Check-out tính tiền → Dọn dẹp phòng → Khóa bảo trì.



Dữ liệu

Thiết kế lược đồ CSDL chuẩn hóa trên MySQL (Engine InnoDB) nhằm bảo đảm tuyệt đối tính chất ACID cho mọi giao tác.

Tính nhất quán ACID



Atomicity & Consistency

Tính nguyên tử: Giao dịch thành công toàn bộ hoặc thất bại hoàn toàn, không có trạng thái lắp lửng.

Tính nhất quán: Chuyển hệ thống từ một trạng thái hợp lệ này sang một trạng thái hợp lệ khác, tuân thủ mọi ràng buộc dữ liệu.



Isolation & Durability

Tính cô lập: Các giao dịch chạy đồng thời không được can thiệp lẫn nhau (kiểm soát qua Isolation Levels).

Tính bền vững: Dữ liệu khi đã được hệ thống commit sẽ được lưu trữ vĩnh viễn dù có xảy ra sự cố đột ngột.

Cơ chế khóa đồng thời



Nền tảng công nghệ:

MySQL 8.0 kết hợp Python Flask (Thin-Backend) và thư viện PyMySQL.



Pessimistic Locking (Khóa bị quan):

Sử dụng lệnh `SELECT ... FOR UPDATE` trên dòng dữ liệu, bắt các luồng khác xếp hàng.



Next-Key Locks:

Cơ chế tinh vi của InnoDB kết hợp Record Lock (khóa bản ghi) và Gap Lock (khóa khoảng trống) xử lý dị thường Phantom Read.

Phân tích nghiệp vụ



1. Lọc phòng trống

Phòng phải có trạng thái vật lý 'Trong' và không nằm trong bất kỳ đơn đặt phòng nào có hiệu lực giao cắt thời gian lưu trú.



2. Đặt phòng an toàn

Kiểm tra và chèn bản ghi mới phải diễn ra nguyên tử (Atomic). Tự động ROLLBACK và báo lỗi nếu có xung đột trùng lịch.



3. Vận hành (In/Out)

Check-in đúng trạng thái; Check-out kết hợp gọi hàm vô hướng tính tiền tự động và xuất bản ghi hóa đơn tài chính.



4. Hủy đơn phạt cọc

Xử lý nghiệp vụ phạt tài chính: Xét điều kiện so sánh ngày hủy và ngày nhận phòng (dưới 3 ngày) để tự động đưa tiền cọc về 0.

Thiết kế CSDL (3NF)

Chuẩn hóa cấu trúc

- **Các bảng thực thể:** LoaiPhong, Phong, KháchHang, DatPhong, HoaDon. Đảm bảo chuẩn 3 (3NF).
- **Ràng buộc bảo vệ lịch sử:** Tất cả khóa ngoại sử dụng `ON DELETE NO ACTION`
- **Ràng buộc miền giá trị (CHECK):**
 - `GiaTieuChuan > 0`, `SucChua > 0`.
 - `TrangThai`: 'Trong', 'Dang_O', v.v.
 - `NgayCheckOut > NgayCheckIn`.



Index và View Báo cáo



Chiến lược Lập chỉ mục

Tối ưu hóa hiệu năng truy vấn bằng các Index chuyên biệt:

- ☐ IX_DatPhong_Concurrency: Hỗ trợ quét nhanh khoảng thời gian chống xung đột lịch.
- ☐ IX_Phong_Status: Tối ưu bộ lọc tìm phòng trống theo phân khúc.



Thiết kế Khung nhìn (View)

View vw_DoanhThuKhachSan thực hiện kết nối (JOIN) 5 bảng dữ liệu nền.
Mục đích: Cung cấp trực tiếp bộ số liệu tài chính sạch, loại bỏ hoàn toàn các câu lệnh JOIN phức tạp khỏi tầng ứng dụng Backend.

Triển khai Backend

Thành phần cốt lõi



Thin-Backend (Python Flask): Chỉ đóng vai trò trung chuyển tham số từ Web xuống Database và ánh xạ mã lỗi ngược lên UI.



Stored Procedures: Đóng gói logic kiểm tra phòng và đặt phòng nguyên tử.



Đồng bộ bằng Trigger: Cập nhật trạng thái vật lý của phòng sang 'Đang sơ' tự động ngay khi đơn hàng chuyển sang 'Hoàn thành'.

```
SELECT COUNT(*) INTO v_ConflictCount
FROM DatPhong
WHERE MaPhong = p_MaPhong AND ...
FOR UPDATE;
```



Dashboard Báo Cáo



Trực quan hóa doanh thu

Biểu đồ tăng trưởng tài chính rõ ràng, tức thì.



Trạng thái phòng real-time

Theo dõi tình trạng phòng (sẵn sàng, bận) theo thời gian thực.

Cơ chế: View dữ liệu sạch

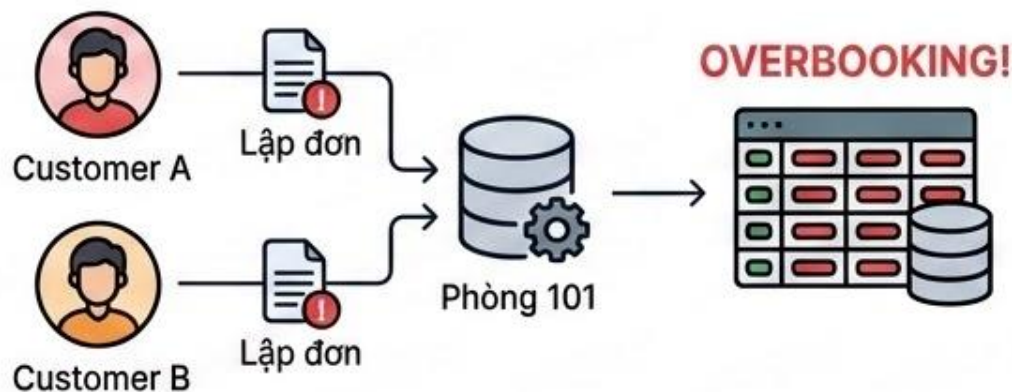


Clean Data View

Chống Đặt trùng phòng

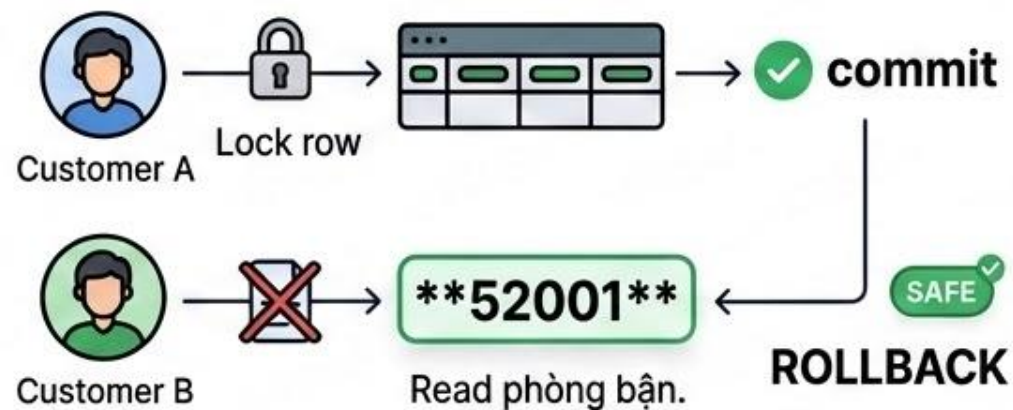
⚠ Tình huống: Lost Update

Hai khách hàng A và B cùng lúc tìm kiếm và thấy phòng 101 đang trống. Cả hai đồng thời phát lệnh đặt phòng. Nếu không kiểm soát, cả hai đơn đều thành công gây ra sự cố **Overbooking** nghiêm trọng.

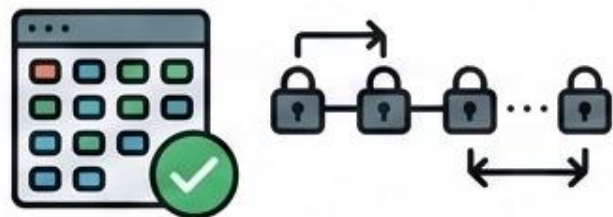


Kết quả an toàn

Stored Procedure kích hoạt mức cô lập **SERIALIZABLE** và lệnh **FOR UPDATE**. Luồng của khách A khóa dòng, luồng B bị block chờ. Sau khi A commit, B đọc lại thấy phòng bận, kích hoạt lỗi **52001** và **ROLLBACK** an toàn.



Phantom Read & Deadlock



Chống Đọc bóng ma

Tình huống: Lễ tân đang đếm báo cáo, khách hàng chèn đơn mới xen vào ngày đó.

Giải quyết: Cơ chế Gap Lock của InnoDB khóa toàn bộ dải chỉ mục ngày. Khách hàng bị chặn lại cho đến khi Lễ tân đọc xong, đảm bảo tính nhất quán tuyệt đối cho báo cáo.



Kiểm soát Khóa chết

Tình huống: Luồng A khóa phòng 1, xin khóa phòng 2. Luồng B khóa phòng 2, xin khóa phòng 1 tạo vòng lặp vô hạn.

Giải quyết: InnoDB tự động phát hiện chu trình Deadlock, chủ động hủy giao dịch tốn ít tài nguyên. Flask bắt lỗi 53001 mượn mà.

Dirty Read & MVCC



Đảm bảo Nhất quán 100%

- ✓ Ngăn chặn việc đọc dữ liệu chưa commit.

100%

Nhất quán Dữ liệu

- ✓ Ngăn chặn việc đọc dữ liệu chưa commit.
- ✓ Kết quả chính xác cho báo cáo và quản lý.



Công nghệ MVCC

- **Tình huống:** Lễ tân tạo hóa đơn 10 triệu VNĐ nhưng chưa commit (chờ ngân hàng). Cùng lúc, quản lý mở Dashboard xem tổng doanh thu.
- **Kết quả:** Thay vì cộng dồn tiền ảo chưa thanh toán hoặc bị block chờ đợi, cơ chế Multi-Version Concurrency Control (MVCC) truy cập **Undo Log**.
- Quản lý đọc được phiên bản dữ liệu sạch lịch sử, đảm bảo số liệu chính xác.

Unit Testing



10/10

Ca kiểm thử ĐẠT (PASS)



Xác minh toàn diện

- Tìm phòng khi có đơn trùng lặp.
- Ngoại lệ đặt phòng (Overbooking).
- Check-in lỗi trạng thái, Check-out gọi Trigger.
- Chính sách tính tiền phạt cọc theo ngày.

Hệ thống hoạt động 100% chuẩn xác theo các ràng buộc thiết kế.

Kết luận & Phát triển



Thành quả

Kiến trúc Thick-Database đã chứng minh hiệu quả cực cao trong việc loại bỏ rủi ro bất đồng bộ, mang lại độ tin cậy vững chắc cho vận hành khách sạn.



Tự động hóa

Tương lai sẽ tích hợp Event Scheduler của MySQL để tự động hóa hoàn toàn chu trình dọn dẹp, chuyển trạng thái phòng sau thời gian quy định.



Mở rộng

Đề xuất kết hợp hệ thống Cache phân tán (Redis) để tăng tốc độ lọc và giảm tải truy vấn đọc trực tiếp dưới MySQL khi lượng truy cập lớn.

